Defining Interfaces

Sometimes the type checking only cares about a subset of the properties of an object, or you have a common set of properties that might be shared by a number of different objects that are otherwise unrelated. It's useful to allow the type checking system to be as specific as possible and to specify that the objects being used are restricted to only the properties that are being used in a specific context.

Alternately, you might have some data in your system, such as the result of a server call that has returned JSON objects, and you'd like to assert in the type system that the certain properties must exist in the data. But you don't necessarily need to declare a class because there might not be any behavior for that data, just a structure you want to use.

You can manage all these issues in TypeScript by using *interfaces*. We've seen this in our React code, where we have been using interfaces to define type information for the props objects passed to each component.

Properties don't have to be just variables; they can also be methods, like `onFilterButtonClick(x: Event): void`. There are more complex scenarios for some rarer items that we won't get into here. As with functions, you can specify a property is optional by replacing the `:` with `?:`. I recommend doing that rarely, as it weakens the type checking.

Once we have this interface, it's just as much a type as any basic type or class. We can specify the interface as the type of a variable, parameter, or return value, and the TypeScript compiler will enforce that those variables only use the listed properties.

You can also specify that a class implements a particular interface, so we could now say this:

```
export default class SeatData implements SeatProps {
  seatNumber: number
```